Министерство науки и высшего образования РФ

Пензенский государственный университет

Кафедра «Вычислительная техника»

Отчёт

По лабораторной работе №7

По дисциплине «Логика и основа алгоритмизации в информационных системах».

На тему «Обход графа в глубину»

Выполнили студенты группы 25ВВВ3:

Ратьков Д. И.

Градсков А.Д

Приняли:

к.т.н., доцент Юрова О.В.

к.т.н., доцент Деев М.В

Пенза 2026

ЛАБОРАТОРНАЯ РАБОТА №7

Обход графа в глубину

Цель работы: научиться выполнять обход графа в глубину для графа, представленного матрицей смежности и списками смежности, а также преобразовывать рекурсивную реализацию обхода в глубину в нерекурсивную.

Практическая часть

Задание 1

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа G. Выведите матрицу на экран.
2. Для сгенерированного графа осуществите процедуру обхода в глубину, реализованную в соответствии с приведенным выше описанием.
3. * Реализуйте процедуру обхода в глубину для графа, представленного списками смежности.

Задание 2*

1. Для матричной формы представления графов выполните преобразование рекурсивной реализации обхода графа к не рекурсивной.

Пояснительная записка к программе

Реализовали программу. Она случайным образом генерирует неориентированный граф G из n вершин (n вводится с клавиатуры), выводит его матрицу смежности и выполняет обход графа в глубину тремя способами: рекурсивно по матрице смежности (задание 1), рекурсивно по спискам смежности (пункт 3*) и нерекурсивно по матрице смежности с использованием собственного стека (задание 2*). Обход запускается из каждой ещё не посещённой вершины, поэтому проходятся все компоненты связности графа. В коде вершины нумеруются с 0 до n−1, а на экран выводятся с 1 до n.

Назначение функций

create_matrix(n) — выделяет память под матрицу смежности n×n в виде двумерного динамического массива int** и заполняет её нулями (calloc). При ошибке выделения памяти освобождает уже выделенные строки и возвращает NULL.

free_matrix(g, n) — освобождает память, занятую матрицей смежности.

generate_graph(g, n) — для каждой пары вершин i < j случайно (rand() % 2) определяет, есть ли между ними ребро, и записывает значение симметрично: g[i][j] = g[j][i]. Диагональ остаётся нулевой, поэтому петель нет.

print_matrix(g, n) — выводит матрицу смежности на экран с номерами строк и столбцов.

dfs_rec(g, n, num, v) — рекурсивная процедура DFS(v) по матрице смежности: помечает вершину v как посещённую (num[v] = 1), выводит её номер, затем просматривает v-ю строку матрицы и для каждой смежной непосещённой вершины i рекурсивно вызывает себя.

dfs_matrix_recursive(g, n) — реализует алгоритм ПОГ: создаёт массив пометок num, заполненный нулями (все вершины «новые»), и для каждой ещё не посещённой вершины вызывает dfs_rec.

free_lists(adj, n) — освобождает память, занятую списками смежности.

matrix_to_lists(g, n) — строит по матрице смежности списки смежности: для каждой вершины i создаётся односвязный список вершин j, для которых g[i][j] = 1. Вспомогательный массив tail хранит указатели на последние элементы списков, чтобы добавлять новые элементы в конец и сохранить порядок возрастания номеров.

print_lists(adj, n) — выводит списки смежности; для вершины без соседей выводится «(нет смежных вершин)».

dfs_list_rec(adj, num, v) — рекурсивный обход в глубину для графа, заданного списками смежности: помечает и выводит вершину v, затем проходит по её списку смежности и рекурсивно запускается из каждой непосещённой соседней вершины.

dfs_lists(adj, n) — полный обход в глубину по спискам смежности (аналог dfs_matrix_recursive).

dfs_matrix_iterative(g, n) — нерекурсивный обход в глубину по матрице смежности. Рекурсия заменена явным стеком stack: на вершине

стека лежит текущая вершина v. Массив pos хранит для каждой вершины номер столбца, с которого нужно продолжить просмотр её строки матрицы. Если найдена смежная непосещённая вершина i, она помечается, выводится и помещается в стек (аналог рекурсивного вызова); если в строке больше нет непосещённых соседей, вершина снимается со стека (аналог возврата из рекурсии). Благодаря pos порядок обхода полностью совпадает с рекурсивной версией.

main() — устанавливает кодировку консоли UTF-8 (в Windows), инициализирует генератор случайных чисел текущим временем, считывает и проверяет число вершин n, создаёт и генерирует граф, выводит матрицу смежности, выполняет рекурсивный обход по матрице, строит и выводит списки смежности, выполняет обход по спискам и нерекурсивный обход по матрице, затем освобождает всю память.

Основные переменные

n — число вершин графа.

g — матрица смежности n×n (двумерный динамический массив int**): g[i][j] = 1, если вершины i и j соединены ребром, иначе 0.

num — массив пометок длиной n (NUM в описании алгоритма): num[v] = 1, если вершина v уже посещена, иначе 0.

v — текущая вершина обхода; i — индекс просматриваемой вершины в строке матрицы.

struct Node — элемент списка смежности: v — номер смежной вершины, next — указатель на следующий элемент.

adj — массив из n указателей на начала списков смежности; tail — массив указателей на последние элементы списков при их построении; p — указатель на текущий элемент списка.

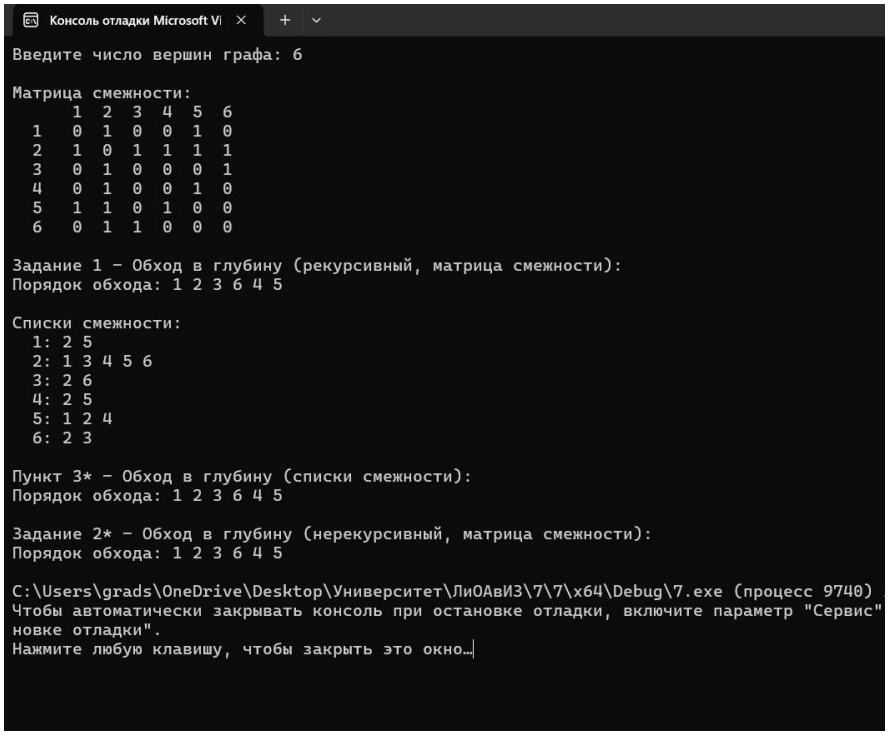
stack, top — массив, используемый как стек вершин в нерекурсивном обходе, и индекс его вершины (top = −1 означает, что стек пуст).

pos — массив позиций: pos[v] — номер столбца в строке v матрицы смежности, с которого продолжится поиск соседей вершины v.

s — вершина, из которой запускается очередной обход в нерекурсивной реализации.

found — флаг: 1, если у текущей вершины найдена непосещённая соседняя вершина, иначе 0.

Результат работы программы:



Все три реализации дали одинаковый порядок обхода 1 2 3 6 4 5, что совпадает с ручной проверкой: из вершины 1 переходим в первую смежную с ней вершину 2, из 2 — в 3, из 3 — в 6; у вершины 6 непосещённых соседей нет, поэтому возвращаемся к вершине 2 и переходим в 4, а из 4 — в 5.

Ссылка на репозиторий: https://github.com/AndreyPayPuv/7lab.git

Вывод: в ходе данной лабораторной работы мы научились выполнять обход графа в глубину для графа, представленного матрицей смежности и списками смежности, а также преобразовали рекурсивную реализацию обхода в нерекурсивную с использованием стека.